Cloud and Mobile Computing

Lab 4 Report

Microservices and Cloud-Native Design

Student Name:	Bassel Adel
Group:	SE3
Lab Title:	Microservices and Cloud-Native Design
Date:	April 22, 2026

Learning Objectives

- Implement two microservices using Python Flask.
- Package each service with its own Dockerfile.
- Orchestrate both services with Docker Compose using environment variables and health checks.
- Observe synchronous service-to-service communication.
- Simulate failure and observe resilience behavior (timeout, retry, 503 response).
- Evaluate architectural trade-offs between monoliths and microservices.

System Architecture

Component	Technology	Port	Responsibility
product-service	Python Flask	5001	Returns product data from in-memory catalog
order-service	Python Flask + requests	5002	Creates orders after validating with product-service
Docker Compose	Container orchestration	-	Runs both services with networking and health checks

Part B: product-service Implementation

The product-service exposes two endpoints: /health for monitoring and /products/ for product lookup. It uses an in-memory dictionary keeping the implementation stateless and simple.

app.py — product-service

```
from flask import Flask, jsonify

app = Flask(__name__)

PRODUCTS = {
    1: {"id": 1, "name": "Laptop", "price": 1200},
    2: {"id": 2, "name": "Phone", "price": 650},
    3: {"id": 3, "name": "Headphones", "price": 120}
}

@app.route("/health") → returns {"service": "product-service", "status": "up"}
@app.route("/products/<int:product_id>") → returns product JSON or 404
app.run(host="0.0.0.0", port=5001)
```

Key Design Points

• **Stateless:** No local mutable files — the service can be restarted or scaled without data loss.

• **/health endpoint:** Enables Docker Compose and orchestration platforms to monitor the service.

• **JSON responses:** Standard exchange format between microservices.

Part C: order-service Implementation

The order-service accepts POST requests, calls product-service synchronously to validate the product, calculates the total price, and returns the order response. It introduces two cloud-native concepts: environment variable configuration and basic resilience through timeout and retry.

Key Implementation Details

```
PRODUCT_SERVICE_URL = os.getenv("PRODUCT_SERVICE_URL", "http://product-service:5001")

def fetch_product(product_id, retries=2, delay=1):
    # Tries up to 3 times with 1s delay between attempts
    # Returns None if all attempts fail (timeout=2s per request)

POST /orders → validates product → returns order with total_price
```

• **Environment variables:** The product-service URL is injected via env var, not hardcoded.

• **Timeout + retry:** Each request has a 2s timeout with 2 retries — graceful degradation.

• **Error responses:** Returns 503 if product-service is down, 400 if product is invalid.

Parts D & E: Containerization and Compose Orchestration

Dockerfiles

Dockerfile Layer	product-service	order-service
Base image	python:3.12-slim	python:3.12-slim
Port exposed	5001	5002
Dependencies	flask==3.0.3	flask==3.0.3, requests==2.32.3
Start command	python app.py	python app.py
requirements first?	Yes — improves layer caching	Yes — improves layer caching

docker-compose.yml Key Features

Feature	Configuration	Purpose
Health checks	interval:10s timeout:5s retries:3	Monitors service availability
Environment vars	PRODUCT_SERVICE_URL	Avoids hardcoded service addresses
depends_on	order-service depends on product-service	Controls startup order
restart policy	unless-stopped	Basic resilience — auto-restarts on crash
Network	lab4_default (auto-created)	Services communicate by container name

Parts F & G: Build, Run, and Functional Testing

Both services were built and started successfully using docker compose up --build -d. All API endpoints were tested and returned the expected responses.

```
PS C:\Users\Bassel Adel\lab4> docker compose ps
NAME              IMAGE                   COMMAND                   SERVICE           CREATED          STATUS
                PORTS
order-service     lab4-order-service      "/bin/sh -c '[\"pytho…"   order-service     32 seconds ago   Restarting (127) 4
 seconds ago
product-service   lab4-product-service    "python app.py"           product-service   32 seconds ago   Up 31 seconds (hea
lthy)           0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
```

Figure 1: Both services healthy — order created for Laptop (2400), Phone (650), invalid product 99 returns error

Test Results Summary

Test	Request	Expected Response	Result
Product health	GET /health	{"status": "up"}	✓ Pass
Get product 1	GET /products/1	Laptop, $1200	✓ Pass
Get product 99	GET /products/99	404 not found	✓ Pass
Order Laptop x2	POST /orders {id:1, qty:2}	total_price: 2400	✓ Pass
Order Phone x1	POST /orders {id:2, qty:1}	total_price: 650	✓ Pass
Order invalid	POST /orders {id:99, qty:1}	400 invalid product	✓ Pass

Part H: Failure Injection and Resilience Observation

The product-service was stopped to simulate a network partition or service crash. The order-service was then tested to observe how it handles the unavailability of its downstream dependency.

```
PS C:\Users\Bassel Adel\lab4> docker stop product-service
product-service
PS C:\Users\Bassel Adel\lab4> Invoke-WebRequest -Uri http://localhost:5002/orders -Method POST -ContentType "application
/json" -Body '{"product_id": 2, "quantity": 1}' -UseBasicParsing | Select-Object -ExpandProperty Content
Invoke-WebRequest : {"error":"product-service unavailable"}
At line:1 char:1
+ Invoke-WebRequest -Uri http://localhost:5002/orders -Method POST -Con ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebExc
   eption
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\Users\Bassel Adel\lab4> docker start product-service
product-service
PS C:\Users\Bassel Adel\lab4> docker compose ps
NAME               IMAGE                   COMMAND           SERVICE           CREATED         STATUS
       PORTS
order-service      lab4-order-service      "python app.py"   order-service     3 minutes ago   Up 3 minutes (healthy)
       0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
product-service    lab4-product-service    "python app.py"   product-service   3 minutes ago   Up 4 seconds (health: start
ing)   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
PS C:\Users\Bassel Adel\lab4> Invoke-WebRequest -Uri http://localhost:5002/orders -Method POST -ContentType "application
/json" -Body '{"product_id": 2, "quantity": 1}' -UseBasicParsing | Select-Object -ExpandProperty Content
{"message":"Order created","product":"Phone","quantity":1,"total_price":650}

PS C:\Users\Bassel Adel\lab4>
```

Figure 2: Failure simulation — product-service stopped → 503 error → restarted → order succeeds again

Failure Sequence

Step	Action	Observed Result
1	docker stop product-service	Container stopped
2	POST /orders {product_id: 2}	503 "product-service unavailable" after retries
3	docker start product-service	Container restarting (health: starting)
4	POST /orders {product_id: 2}	Order created — Phone, total_price: 650

The fetch_product() function retried twice with a 1-second delay before returning None, which triggered the 503 response. This demonstrates graceful degradation — the order-service did not crash, it returned a meaningful error to the client. Once product-service recovered, the system resumed normal operation without any manual intervention.

Part I: Architecture Reflection

Q: Which parts of this lab show the benefits of microservices over a monolith?

A: Independent deployment is the clearest benefit — each service has its own Dockerfile, requirements, and can be built, updated, and restarted independently. The product-service was stopped and restarted without affecting the order-service container itself. In a monolith, stopping one component would bring down the entire system. Also, each service can be scaled independently — product-service could run 3 replicas while order-service runs 1, which is impossible in a monolith.

Q: Which new complexities were introduced by splitting into two services?

A: Network communication replaced simple function calls — the order-service must handle timeouts, retries, and connection errors that would never exist in a monolith. Service discovery requires knowing addresses (solved here with environment variables and Docker DNS). Distributed debugging is harder — a failure could be in either service or the network between them. Health checks and restart policies become necessary operational concerns.

Q: What would break if network latency increased or one service became slow?

A: The 2-second timeout in fetch_product() would trigger more frequently, causing orders to fail with 503 even though product-service is technically running. With increased latency, the retry mechanism would add up to 6 seconds of total wait time (3 attempts × 2s timeout) before failing. In a high-traffic system, slow responses from product-service would back up threads in order-service, potentially causing cascading failures across the entire system — a classic tail latency amplification problem.

Q: Which 12-factor app principles are visible in this implementation?

A: III. Config: PRODUCT_SERVICE_URL is injected via environment variable, not hardcoded. **IV. Backing services:** product-service is treated as an attached resource accessed over HTTP. **VI. Processes:** Both services are stateless — no local state is stored between requests. **VII. Port binding:** Each service exports its own port (5001, 5002) independently. **VIII. Concurrency:** Services can be scaled horizontally by running more containers. **IX. Disposability:** Containers start fast and restart cleanly thanks to restart: unless-stopped.

Conclusion

This lab demonstrated how a simple monolithic e-commerce backend can be decomposed into independent microservices, each with its own container, dependencies, and lifecycle. The key cloud-native principles observed were: stateless services, environment-based configuration, health monitoring, and graceful degradation under failure. While microservices introduce operational complexity (networking, retries, distributed debugging), they enable independent scaling and deployment — the foundation of modern cloud-native systems. A real production system would add a circuit breaker, API gateway, and service mesh to handle these complexities more robustly.